Web Services and Web Data Coursework 1

Project Title: RailPulse

Student Name / ID: Alexander East / 201612203

Deliverables:

- **GitHub Repository:** https://github.com/mirriz/railpulse
- **Live API (Render):** https://railpulse-w5g2.onrender.com/docs
- **Live Frontend (Render):** https://railpulse-front.onrender.com/
- **API Documentation (SwaggerUI):** https://github.com/mirriz/railpulse/blob/main/documentation/railpulse_api_doc_swagger_expanded.pdf
- **PowerPoint Slides:** https://docs.google.com/presentation/d/1mmNmFScT_AxQblmeGjL9yJJFjcoG2BKT/edit?usp=drive_link&ouid=116388432257716081540&rtpof=true&sd=true

GenAI Declaration: As per the “Green” assessment category, I acknowledge the use of Generative AI (specifically, Gemini and Copilot) in the planning, structure, and implementation of my work.

1. Executive Summary

Having experience working within the transport sector at Network Rail, it was made clear that business decisions were driven by two Key Performance Indicators (KPIs); reducing train delays and improving customer satisfaction. However, current transport API solutions treat these KPIs as separate issues. They report raw delay data live but fail to capture the in-the-moment passenger experience, such as overcrowding. Because of this blind spot, operators can struggle to measure how delay metrics translate to passenger approval.

RailPulse was created to bridge this gap. It is a crowdsourced analytics API that moves away from traditional train-to-train delay reporting toward a **Hub Health Model**. By fusing real-time National Rail telemetry via the Huxley 2 API (Singleton, J, 2024) with subjective passenger sentiment reports, RailPulse generates a real-time **Stress Index** (0.0 - 1.0). This provides operators with a unified reflection of station health and all train services operating within them. This approach delivers business value, as rather than chasing delayed trains across a map, operators can pinpoint the originating bottleneck stations where passenger friction occurs. By exposing these issues in real-time, control rooms can deploy immediate interventions that mitigate cascading network delays and passenger complaints.

2. Architectural Design and Stack Justification

To ensure the system architecture met enterprise standards for an analytics tool, Generative AI was used extensively during the initial ideation phase to reimagine the project scope. As shown in Figure 1 below, rather than building a delay-tracking application, the AI was prompted to generate high level concepts instead. The AI’s option B: “StationFlow" inspired RailPulse’s pivot toward a station-centric hub health model. This shift decided the technical requirements for the rest of the stack; the system required a concurrent framework to handle live data ingestion and a relational database to manage crowdsourced state.

To begin stack selection, *FastAPI* was selected over *Flask* or *Django* due to the input-output nature of the application. RailPulse relies on real-time data ingestion from the *Huxley 2 API* (a proxy for

National Rail feeds). Flask's architecture would block the main thread during these external network calls, leading to latency under load. FastAPI's implementation and *asyncio* support allow for non-blocking network requests, allowing high concurrency (FastAPI, 2024). In addition, use of generative AI uncovered that FastAPI integrates with *Pydantic*, providing automatic request/response data validation and generation of *OpenAPI* documentation directly from the code, satisfying the requirement for quality API contracts (Pydantic, 2024).

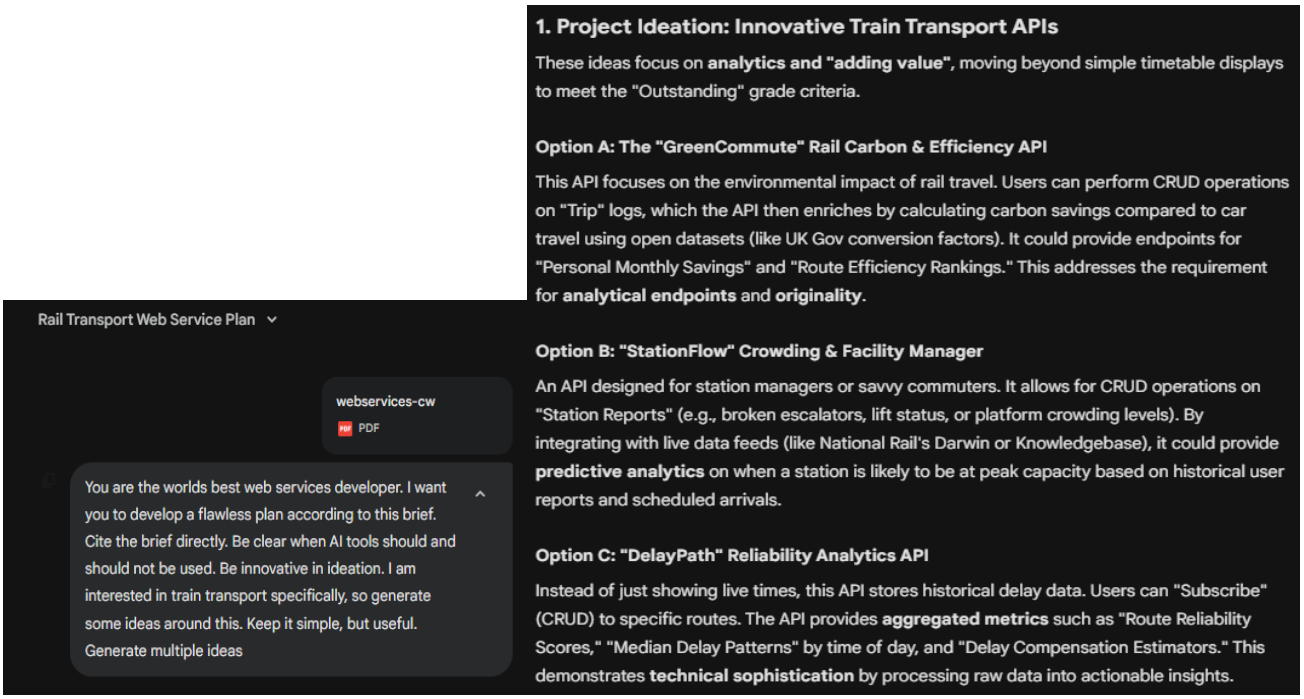

Figure 1 – The initial planning and ideation prompt and response

For data storage, while a local *SQLite* database would work for a development environment, it is less compatible with the chosen cloud hosting solution, *Render*, which wipe local files on redeployment. Therefore, a cloud database was mandatory. Azure Database for PostgreSQL was selected to simulate an enterprise production environment (Microsoft, 2026). A relational SQL database was chosen over a NoSQL alternative to enforce relational integrity. The core security mechanism of RailPulse, Role Based Access Control, relies on a foreign key relationship linking the users table (*owner_id*) to the incidents table. PostgreSQL guarantees this referential integrity at the database level, ensuring incidents with no owner cannot exist and users can only manipulate their own data.

To prove RailPulse as an enterprise-grade API engine, a dynamic frontend was built using *Streamlit* (Snowflake Inc, 2026). Rather than coupling the UI to FastAPI, the frontend is deployed as an isolated cloud microservice. It uses the FastAPI endpoints via standard RESTful HTTP requests and token exchanges. This decision demonstrates true separation of items; the API remains stateless and can scale to serve multiple clients, such as the frontend, simultaneously without friction.

3. API Functions

The RailPulse API uses a RESTful interface divided into three operational domains. The identity section (*/users*) provides scalable authorisation via signed JSON Web Tokens (JWTs) and *bcrypt* credential hashing (Python Cryptographic Authority Developers, 2025). This security model protects the state management layer (*/incidents*), providing an authenticated CRUD interface for real-time passenger sentiment reporting. This ingestion point is protected by *Role-Based Access Control* (RBAC), guaranteeing that users can only edit their own cryptographic data. Serving as the

system's intelligence engine is the analytics layer (*/analytics* & */live*). This domain acts as a proxy for live National Rail Darwin feeds while operating as the execution environment for the proprietary stress index algorithm. By fusing the objective telemetry with the internal subjective state gathered from user reports, this layer generates the station-level analytics that define the system's novel data integration.

Initially, the prototype was constrained to a single, hardcoded transport hub, Leeds. To elevate the project to an outstanding, production-ready standard, Generative AI suggested to create a scaling strategy (see Figure 3). Acting on this consultation, the API and Database were edited to support dynamic routing where incident reports were linked to station identifiers, allowing the API to scale nationally instead of being region locked.

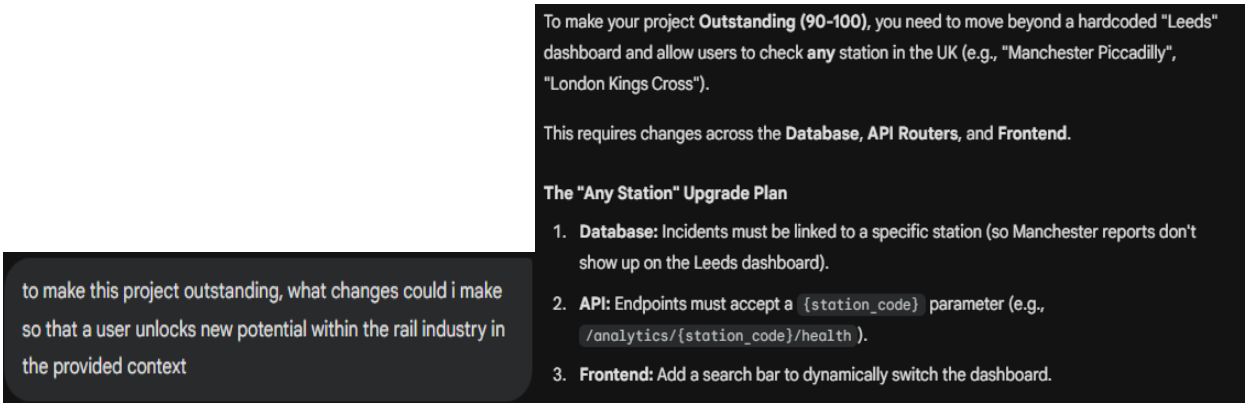

Figure 2 – GenAI for novel idea generation, integrated into final product

4. Novel Data Integration

While traditional rail systems use route reliability scores, RailPulse pivots this concept to generate real-time station-level reliability scores. Rather than outputting raw delay minutes, the system performs real-time data analytics; it aggregates objective National Rail telemetry data (ingested via the Huxley 2 API) with subjective user incident reports submitted within the last hour. Fusing this objective time data with subjective categorical data presented a structural challenge. To achieve algorithmic complexity, generative AI was consulted (see Figure 2) to help formulate a normalisation strategy. The resulting algorithm, executed in *src/routers/analytics.py*, structures these data types into a unified 0.0-1.0 stress index. It applies a 60% weight to live train delays and a 40% weight to crowdsourced incident severity ratings. This turns JSON responses into actionable and operational metrics that represent the status of any station UK-wide. In addition, there was a flaw in the standard mathematical averaging algorithm that was being used; a station with zero delayed trains but 50% cancelled trains could register as perfectly on time with 0 minutes average delay.

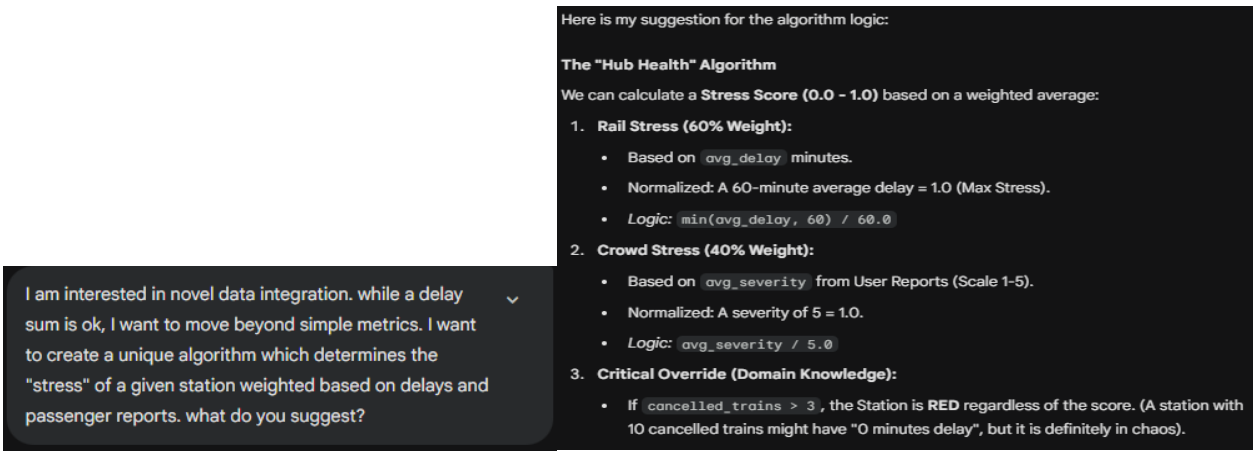

Figure 3 – GenAI suggestion for advancing complexity

To correct this distortion, an override was built into the analytics router. As shown in Figure 3 and expanded in the final deployment, the algorithm assesses the total ratio of cancelled services, and if cancellations are deemed excessive, the algorithm forcefully bypasses the mathematical average and escalates the status to "RED" or “AMBER”. This ensures the API does not output a naive mathematical average, but rather an accurate reflection of the true passenger experience and overall network status.

5. Security and Access Control

When a user creates a RailPulse account, their password is hashed using *bcrypt* before being saved, meaning even database administrators cannot read it. When a user logs in successfully, instead of the server trying to remember their user state, the system hands them a token called a JSON Web Token (JWT). The user presents this token every time they want to use the app. Because the server doesn't have to waste memory saving who is logged in, the application can easily run faster and handle thousands of users at once, improving scalability. If someone tries to access a protected part of the app without a valid token, the system blocks them with a 401 unauthorized error.

Beyond just logging in, the system makes sure that users strictly own the data they create using Role-Based Access Control (RBAC). When a user submits a crowd-sourced report about a station, the database permanently links their unique user ID to that specific report. Later, if someone tries to edit or delete that report, the system double-checks their digital ID and will only permit the change if the user making the request is the same person who originally wrote it. If another user or a malicious actor tries to tamper with someone else's report, they are blocked with a 403 forbidden error, standard web code practise. This ensures nobody can vandalise other user’s data, keeping the final stress index score more trustworthy than if this were the case.

6. Testing and Cloud Deployment

RailPulse uses an automated testing suite by utilising *pytest* (Pytest, 2026). To ensure no data pollution in the live database environment, the testing architecture leverages FastAPI’s dependency injection framework to override the primary *get_db* database generator. A custom conftest.py fixture dynamically allocates an isolated, in-memory SQLite database that is created and destroyed for individual tests. This prevents state leakage between tests, ensuring true test isolation and deterministic results.

To ensure full test coverage of the system's security and logic, generative AI was leveraged as a quality assurance tool (see Figure 4). By prompting the AI with the system's RBAC constraints and framework, it generated a production-ready testing plan. This directly lead the implementation of edge-case tests, specifically boundary testing the RBAC implementation (verifying 403 forbidden responses upon unauthorised edit attempts) and validating data integrity (ensuring local station reports do not leak into other station analytics).

The API is deployed as a cloud application hosted on Render (Render, 2026), interfacing with an enterprise-grade Azure Database for PostgreSQL. During deployment, a challenge arose regarding network security; Render's dynamic IP addressing conflicted with Azure’s default firewall policies. To ensure connectivity for this prototype, the Azure firewall was configured to accept all incoming IP addresses (0.0.0.0 to 255.255.255.255). However, as a reflection, in a production environment, this security compromise could be resolved by using *a Virtual Private Cloud* (VPC), securing the application without exposing the database ports to the public internet.

mirriz/railpulse

GitHub

You are a Senior QA Automation Engineer. I am building a FastAPI backend that features JWT-based authentication, an /incidents router protected by Role-Based Access Control (RBAC) where users can only edit their own crowdsourced station reports, and a public /analytics router that calculates an index for UK train stations. See the repo attached for full details.

Please outline a comprehensive testing plan. Detail the exact test functions I need to write to ensure robust coverage. Group them by logic and include edge cases where a system may fail in odd places.

Phase 1: Authentication & Identity Security

First, we must ensure the cryptographic gates are functioning correctly and that the database handles constraints.

- `test_register_user_a` : Verify standard 201 creation and ID generation.
- `test_register_user_b` : Provision a secondary user for later RBAC cross-contamination testing.
- `test_register_duplicate_email` : **Edge Case** - Ensure the DB raises a 400 error and prevents duplicate identities.
- `test_login_success` : Verify valid credentials return a well-formed JWT Bearer token.
- `test_login_failure` : **Edge Case** - Verify unauthorized 401 response on bad passwords.

Phase 2: Protected CRUD Operations (Incidents)

These tests must utilize the JWT token generated in Phase 1 to verify state manipulation.

- `test_create_incident_default_station` : Verify a user can post a report and the system defaults to "LDS" if omitted.
- `test_create_incident_manchester` : Verify explicit station code assignment ("MAN").

Phase 5: API Validation Edge Cases

Testing how the API handles malformed client requests.

- `test_delete_non_existent_incident` : **Edge Case** - Send a valid UUID that does not exist in the DB. Must return 404 Not Found, not a 500 Server Error.
- `test_validation_missing_fields` : **Edge Case** - Send a POST request missing a required Pydantic field (e.g., `severity`). Verify FastAPI intercepts it and returns a 422 Unprocessable Entity.

Figure 4 – GenAI for generating test cases and examining edge scenarios

7. Potential Challenges and Reflection

While RailPulse demonstrates complex data integration, the current architecture possesses a potential scalability bottleneck regarding its reliance on the Huxley 2 API. Currently, every client request to *the /analytics/{station_code}/health* endpoint triggers a live network call to the National Rail proxy. While FastAPI prevents thread-blocking during this I/O operation, the system remains reliant on Huxley's latency and rate-limiting protocols. If the application experienced a sudden spike in traffic, RailPulse would barrage the Huxley API with many requests, resulting in potential rate-limiting or upstream server timeouts. To elevate this prototype to a true production environment, the priority would be the implementation of a caching layer, such as Redis. By caching the external Huxley payloads for a short *Time-To-Live* (TTL) of for example 60 seconds, the API could serve thousands of concurrent requests directly from memory rather than triggering redundant external HTTP calls.

Furthermore, while PostgreSQL is the optimal choice for the current MVP, it is not purpose-built for high velocity temporal data. If RailPulse were to expand beyond real-time analytics into historical trend forecasting such as allowing operators to analyse a station's stress over a multi-year period, the incidents table would bloat massively. A future iteration of the architecture would benefit from migrating the analytical metrics to a *Time-Series Database* (TSDB), separating the heavy analytical queries from the transactional database to preserve API performance.

Overall, RailPulse is a success, delivering a secure, enterprise-grade prototype that fuses live train telemetry with crowdsourced sentiment to manage key UK rail KPIs in a unique fashion. The next step would be to implement a caching layer to handle high-traffic demands, leading the way to scale this new hub health system across the UK train network

References

FastAPI (2024). ‘FastAPI Documentation’. Available at: https://fastapi.tiangolo.com/ (Accessed: 1 March 2026).

Microsoft (2026). ‘Azure Database for PostgreSQL’. Available at: https://azure.microsoft.com/en-us/products/postgresql (Accessed: 3 March 2026).

Pydantic (2024). ‘Pydantic’. Available at: https://docs.pydantic.dev/latest/ (Accessed: 1 March 2026).

Pytest (2026). ‘pytest: helps you write better programs’. Available at: https://docs.pytest.org/en/stable/ (Accessed: 3 March 2026).

Python Cryptographic Authority Developers (2025). ‘bcrypt’. Available at: https://pypi.org/project/bcrypt/ (Accessed: 3 March 2026).

Render (2026). ‘Render Documentation’. Available at: https://render.com/docs (Accessed: 3 March 2026).

Singleton, J (2024). ‘Huxley 2: Community Edition’. Available at: https://huxley2.azurewebsites.net/ (Accessed: 1 March 2026).

Snowflake Inc. (2026). ‘Streamlit Documentation’. Available at: https://docs.streamlit.io/ (Accessed: 5 March 2026).

I acknowledge the use of Copilot (Microsoft, https://m365.cloud.microsoft/chat) to support in ideation, implementation, and testing of my coursework.

I acknowledge the use of Gemini (Google, https://gemini.google.com) to support in ideation, implementation, and testing of my coursework.

I acknowledge the use of Gamma (Gamma, https://gamma.app/) to support in ideation, implementation, and testing of my coursework.